

Modular STM32-Based CAN Telemetry System for IMU Logging, RTC Timestamping, and Offline Orientation Visualisation

Yahia Eid
ye2g24@soton.ac.uk

Abstract: This report presents a modular STM32-based telemetry system for inertial measurement acquisition, CAN communication, local data logging, and offline analysis. The system consists of a Vehicle Dynamics Node and a Logger Node implemented using STM32 NUCLEO-F446RE development boards. The Vehicle Dynamics Node acquires three-axis accelerometer and gyroscope data from an MPU-6050 inertial measurement unit over I2C and transmits the readings over a physical CAN bus using standard CAN frames. The Logger Node receives and decodes the CAN frames, timestamps each sample using a DS3231 real-time clock, and stores the resulting telemetry data on a microSD card in CSV format. The development process included MPU-6050 self-test, CAN loopback validation, RTC integration, SPI-based microSD/FatFS storage, and final two-node CAN-to-SD logging. The logged data was imported into MATLAB, where raw IMU readings were converted into physical units, plotted over time, and used to estimate roll and pitch from the accelerometer gravity vector. An offline MATLAB dashboard was developed to replay the estimated node orientation using a 3D board model with time-slider playback. Experimental testing confirmed successful CAN transmission, timestamped CSV logging, and qualitative agreement between physical board motion and the MATLAB orientation replay.

1. Introduction

Modern vehicle and motorsport systems rely on telemetry to observe how a system behaves during testing. Measurements such as acceleration, angular velocity, temperature, speed, and subsystem status can be collected from distributed electronic nodes and used for later analysis. In embedded vehicle systems, Controller Area Network (CAN) is commonly used to connect multiple nodes over a shared communication bus, making it suitable for modular telemetry architectures.

The aim of this project was to design and implement a prototype STM32-based telemetry system that demonstrates the main stages of an embedded data acquisition pipeline. The system was developed around a Vehicle Dynamics Node and a Logger Node. The Vehicle Dynamics Node is responsible for acquiring inertial measurement data, while

the Logger Node is responsible for receiving telemetry messages and storing them for later analysis.

The project focuses on practical embedded systems development rather than producing vehicle-qualified hardware. The main engineering goals were to acquire sensor data from an IMU, transmit the data between separate STM32 nodes over CAN, timestamp and store the received telemetry locally, and post-process the logged data using MATLAB. This provides an end-to-end prototype covering sensor interfacing, bus communication, data logging, and offline visualisation.

This report presents the system architecture, hardware design, firmware implementation, CAN communication protocol, logging method, MATLAB post-processing, test results, limitations, and possible future extensions of the telemetry system.

2. System Overview

The telemetry system was designed as a modular two-node embedded architecture for acquiring, transmitting, logging, and analysing inertial measurement data. The system is divided into a Vehicle Dynamics Node and a Logger Node, with MATLAB used as the offline analysis environment. This section introduces the overall structure of the system, while detailed hardware design, firmware implementation, CAN message formatting, logging, and MATLAB processing are discussed in later sections.

2.1 System Architecture

The high-level architecture of the telemetry system is shown in Fig. 1. The Vehicle Dynamics Node is built around an STM32 NUCLEO-F446RE board connected to an MPU-6050 inertial measurement unit [1], [3]. This node is responsible for acquiring motion data and transmitting it to the rest of the system.

The Logger Node is built around a second STM32 NUCLEO-F446RE board. It is connected to a CAN transceiver, DS3231 real-time clock, and microSD card module [5], [6], [7]. This node acts as the central logging unit, receiving telemetry messages and storing them for later analysis.

The two STM32 nodes communicate through a physical CAN bus using SN65HVD230 CAN transceiver modules. This keeps the sensing and logging functions separate and allows the system to be expanded later by adding further CAN-based sensor nodes.

2.2 Node Responsibilities

Each part of the system was assigned a specific role to keep the design modular. The main responsibilities of each block are summarised in Table I.

Table I: Main responsibilities of each telemetry system block.

System Block	Main Responsibility
Vehicle Dynamics Node	Acquires accelerometer and gyroscope data from the MPU-6050 IMU and transmits the measurements over CAN.
CAN Bus	Provides the communication link between the Vehicle Dynamics Node and the Logger Node.
Logger Node	Receives CAN frames, decodes IMU values, timestamps samples, and stores telemetry data on a microSD card.
DS3231 RTC	Provides real-world time and date information for logged telemetry rows.
microSD Card	Stores telemetry data in CSV format for later analysis.
MATLAB Analysis	Imports the CSV file for plotting, unit conversion, and offline orientation replay.

This separation made it possible to develop and test each subsystem independently before final integration.

2.3 Telemetry Data Flow

The telemetry data flow is shown in Fig. 2. During operation, the Vehicle Dynamics Node reads raw accelerometer and gyroscope samples from the IMU and sends them as CAN messages. The Logger Node receives these messages, decodes the sensor values, adds timing information, and writes each complete sample to a CSV file.

After testing, the generated LOG.CSV file is transferred to a laptop and imported into MATLAB. The logged data is then used for signal plotting, unit conversion, and offline visualisation of the node orientation.

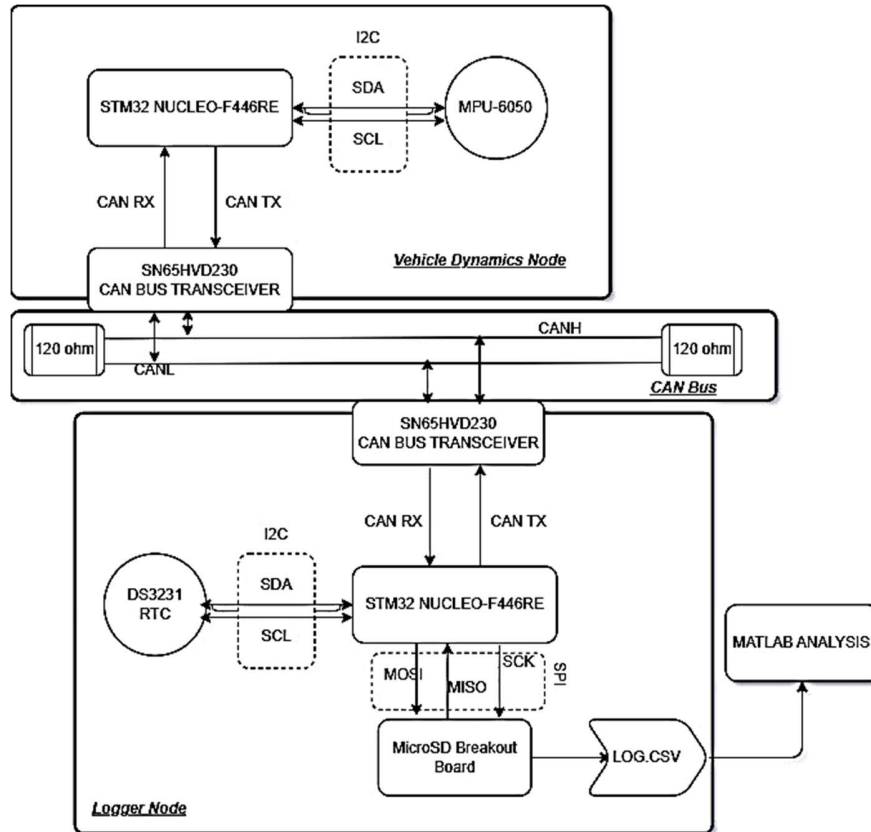


Figure 1: High-level architecture of the modular STM32 CAN telemetry system.

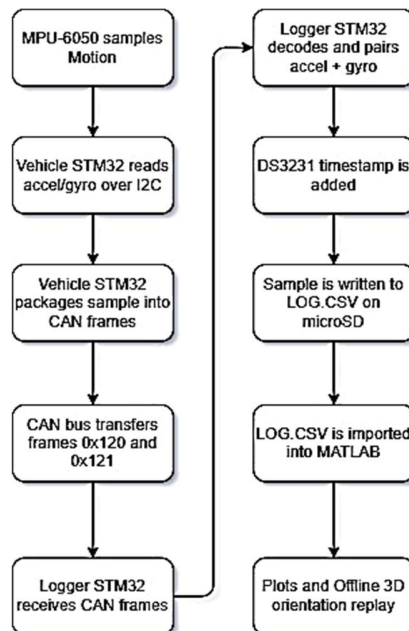


Figure 2: Telemetry data flow from IMU acquisition to CAN transmission, SD card logging, and MATLAB offline analysis

3. Hardware Design

The prototype hardware was divided between the Vehicle Dynamics Node and the Logger Node. Both nodes used STM32 NUCLEO-F446RE development boards, with external modules added for sensing, CAN communication, timestamping, and data storage. The hardware was assembled on breadboards to allow rapid modification during testing and debugging.

3.1 Vehicle Dynamics Node Hardware

The Vehicle Dynamics Node hardware consisted of an STM32 NUCLEO-F446RE board, an MPU-6050 IMU module, and an SN65HVD230 CAN transceiver. The MPU-6050 was used to measure three-axis acceleration and three-axis angular velocity [3]. It was connected to the STM32 through the I2C bus using SDA and SCL lines, and its accelerometer and gyroscope measurement registers were read according to the register map [4].

The MPU-6050 interrupt pin was also connected to the STM32. This allowed the firmware to detect when a new sensor sample was available instead of relying only on continuous polling.

The SN65HVD230 transceiver provided the physical CAN interface for the node. The STM32 CAN TX and CAN RX pins were connected to the transceiver, and the transceiver output was connected to the shared CANH and CANL bus.

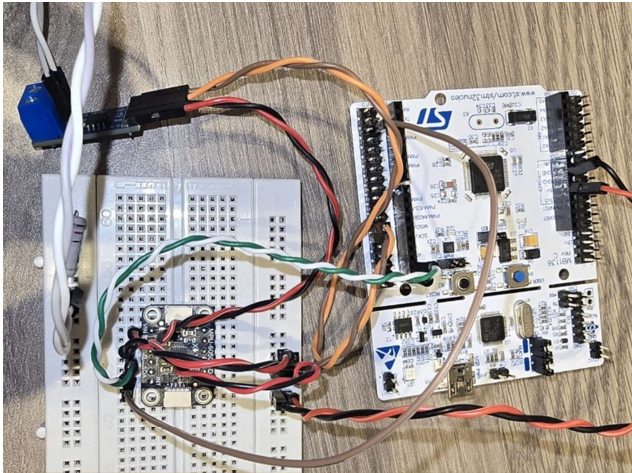


Figure 3: Vehicle Dynamics Node hardware showing the STM32 NUCLEO-F446RE, MPU-6050 IMU, and SN65HVD230 CAN transceiver.

The wiring can be decoded and understood using Table II below:

Table II: Vehicle Dynamics Node physical connections

From	Wire	To
STM32 3V3	Red	SN65HVD230 VCC, MPU-6050 VCC
STM32 GND	Black	SN65HVD230 GND, MPU-6050 GND
STM32 PB8/ I2C1 SCL	Green	MPU-6050 SCL
STM32 PB9/ I2C1 SDA	White	MPU-6050 SDA
MPU-6050 INT	Brown	STM32 PA10/ GPIO
STM32 PA12/ CAN1 TX	Brown	SN65HVD230 CAN TX
STM32 PA11/ CAN1 RX	Orange	SN65HVD230 CAN RX
SN65HVD230 CANH	White	CAN bus CANH / 120 Ω termination
SN65HVD230 CANL	Grey	CAN bus CANL / 120 Ω termination

3.2 Logger Node Hardware

The Logger Node hardware consisted of a second STM32 NUCLEO-F446RE board, an SN65HVD230 CAN transceiver, a DS3231 real-time clock module, and a microSD card breakout board.

The DS3231 RTC was connected over I2C and provided real-world date and time information for logged samples [6], whereas the microSD card breakout board was connected over SPI and provided removable local storage for the generated telemetry file [7], [8].

The CAN transceiver connected the Logger Node to the shared CAN bus. The DS3231 RTC was connected over I2C and provided real-world date and time information for logged samples. The microSD card breakout board was connected over SPI and provided removable local storage for the generated telemetry file.

A card-detect pin from the microSD module was also connected to the STM32. This allowed the firmware to check whether a card was inserted before attempting to access the filesystem.

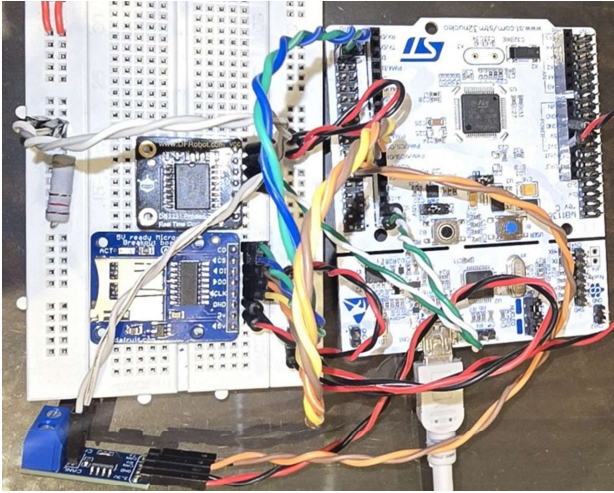


Figure 4: Logger Node prototype showing the STM32 NUCLEO-F446RE, DS3231 RTC module, microSD breakout board, SN65HVD230 CAN transceiver, and CAN bus termination used during integration testing.

The wiring can be decoded and understood using Table III below:

Table III: Logger Node physical connections

From	Wire	To
STM32 3V3	Red	DS3231 VCC, microSD VCC, SN65HVD230 VCC
STM32 GND	Black	DS3231 GND, microSD GND, SN65HVD230 GND
STM32 PB8 / I2C1 SCL	Green	DS3231 SCL
STM32 PB9 / I2C1 SDA	White	DS3231 SDA
STM32 PA5 / SPI1 SCK	Yellow	microSD CLK

From	Wire	To
STM32 PA6 / SPI1 MISO	Orange	microSD DO
STM32 PA7 / SPI1 MOSI	Brown	microSD DI
STM32 PA10 / GPIO	Blue	microSD CS
STM32 PC4 / GPIO	Green	microSD CD
STM32 PA12 / CAN1 TX	Brown	SN65HVD230 CAN TX
STM32 PA11 / CAN1 RX	Orange	SN65HVD230 CAN RX
SN65HVD230 CANH	White	CAN bus CANH / 120 Ω termination
SN65HVD230 CANL	Grey	CAN bus CANL / 120 Ω termination

3.3 Prototype Assembly and Power

During bench testing, the STM32 boards were powered through USB. The external modules were powered from the STM32 supply pins, and a common ground connection was maintained between both nodes. This ensured that the CAN transceivers and digital interfaces shared a common electrical reference. During integration testing, the CANH and CANL wires were twisted together to reduce noise pickup and improve signal integrity in the breadboard prototype.

The breadboard implementation was suitable for controlled prototype testing. A more robust version would require soldered connections or a PCB, secure connectors, strain relief, protected power input, and an enclosure.

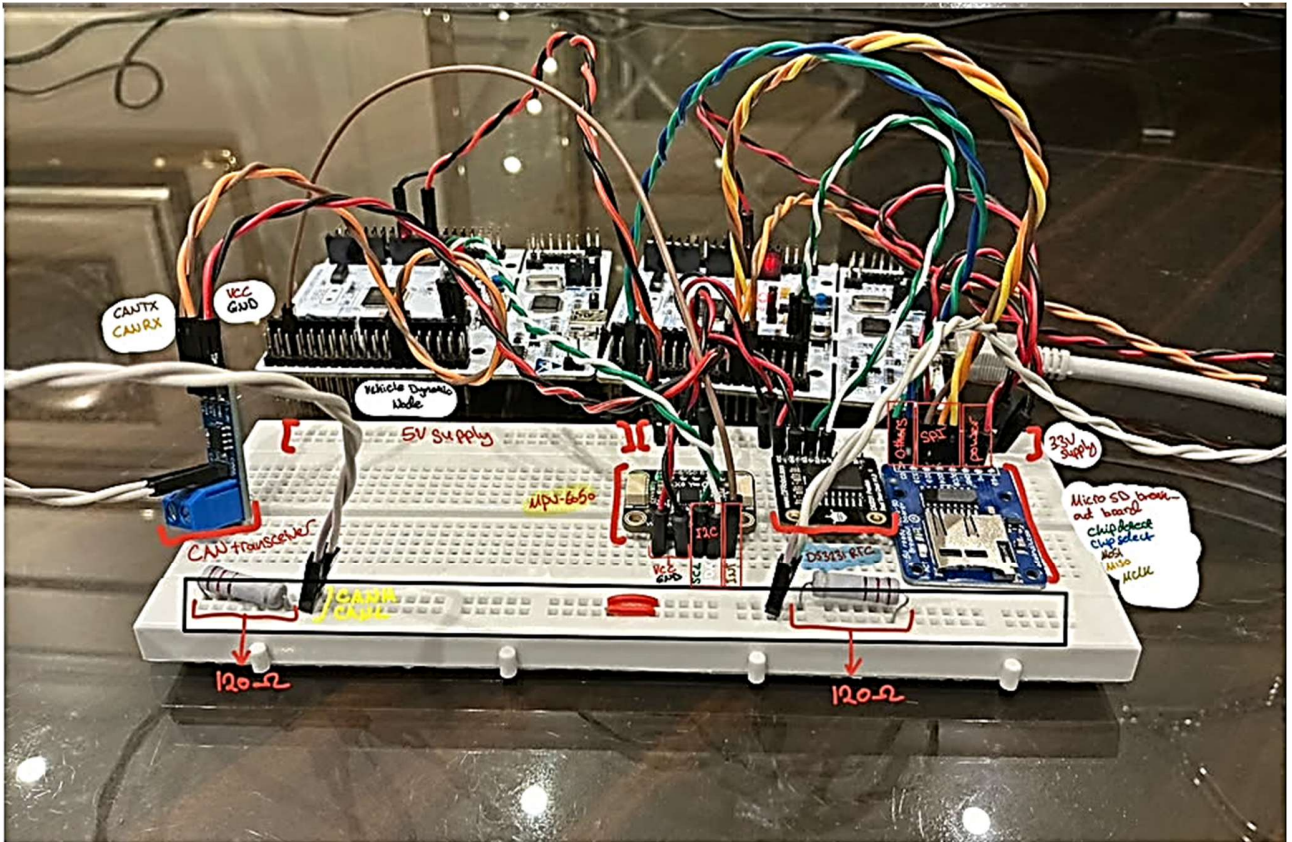


Figure 5: Complete breadboard prototype of the two-node STM32 CAN telemetry system during integration testing, showing the Vehicle Dynamics Node, Logger Node, shared CAN bus wiring, and 120 Ω bus termination.

4. Firmware Design

The firmware was split into two STM32CubeIDE projects, one for the Vehicle Dynamics Node and one for the Logger Node. Each project was generated from STM32CubeMX and implemented using the STM32 HAL libraries.

The Vehicle Dynamics firmware handles IMU configuration, sample acquisition, and CAN transmission. The Logger firmware handles CAN reception, RTC timestamping, SD card mounting, and CSV file writing. This section summarises the final firmware structure and operating logic; detailed code listings and debugging notes are provided in the accompanying development logbook.

4.1 Vehicle Dynamics Node Firmware

The Vehicle Dynamics Node firmware begins by initialising the STM32 peripherals required for IMU acquisition and CAN transmission. These include I2C for the MPU-6050, CAN for telemetry transmission, GPIO for the MPU-6050 interrupt pin, and UART for debugging.

After startup, the MPU-6050 is configured for normal telemetry operation. The device is woken from sleep mode, the digital low-pass filter is enabled, the sample rate is configured, and the accelerometer and gyroscope ranges are set. The MPU-6050 data-ready interrupt is then enabled so that the STM32 can detect when a new sample is available [4].

When the data-ready interrupt occurs, the interrupt callback sets a software flag rather than reading the sensor immediately. The main loop checks this flag, reads the accelerometer and gyroscope registers over I2C, packages the readings into CAN frames, and transmits the frames using the STM32 CAN peripheral. This keeps the interrupt routine short and avoids performing blocking I2C or CAN operations inside the interrupt.

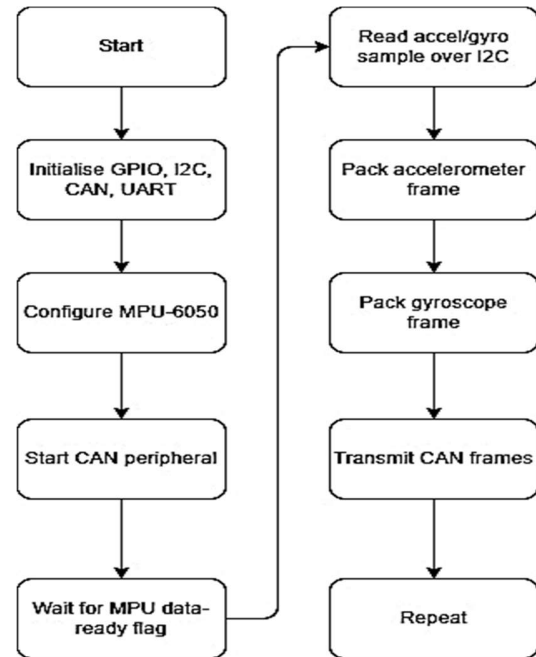


Figure 6: Vehicle Dynamics Node firmware flow from MPU-6050 data-ready interrupt to CAN frame transmission.

4.2 Logger Node Firmware

The Logger Node firmware used the STM32 CAN, I2C, SPI, GPIO, and UART peripherals configured through STM32CubeMX and implemented using the STM32 HAL libraries [2].

At startup, the firmware initialises the CAN peripheral and configures a receive filter so that CAN frames can be received into FIFO0. The DS3231 RTC is also initialised and later used to provide a real-world timestamp for each logged sample. Before attempting file operations, the firmware checks the microSD card-detect pin and mounts the FAT32 filesystem using FatFS.

During operation, the Logger Node polls the CAN receive FIFO. When an accelerometer frame is received, the three accelerometer values are decoded and stored. When a gyroscope frame is received, the three gyroscope values are decoded and stored. Once both frame types have been received, the firmware treats the data as a complete IMU sample. A timestamp is read from the DS3231 RTC, the current system tick is also recorded, and the complete sample is appended to LOG.CSV on the microSD card.

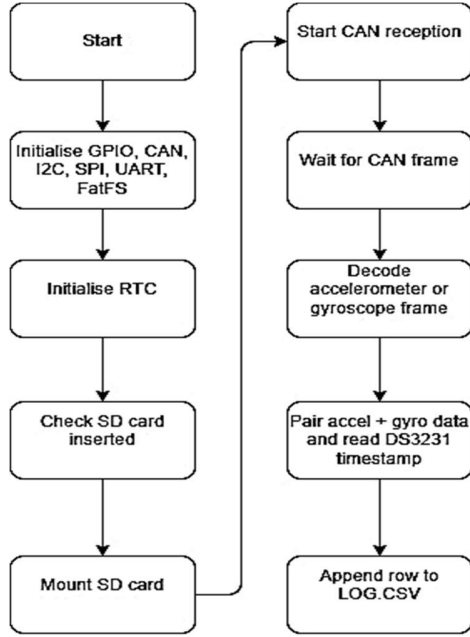


Figure 7: Logger Node firmware flow from CAN frame reception to timestamped CSV logging.

Table IV: Main firmware functions used in the telemetry system

Function	Node	Purpose
MPU6050_normal_init()	Vehicle Dynamics	Configures the MPU-6050 for normal accelerometer and gyroscope telemetry.
MPU6050_read_sample()	Vehicle Dynamics	Reads accelerometer and gyroscope data from the MPU-6050 over I2C.
CAN_send_imu_frames()	Vehicle Dynamics	Packages IMU values into CAN frames and transmits them.
CAN_Logger_Init()	Logger	Configures and starts CAN reception on the Logger Node.
DS3231_GetTimestamp()	Logger	Reads the RTC and formats the timestamp string.
Log_IMU_Row()	Logger	Writes a complete timestamped IMU sample to LOG.CSV.
Mount_SD_With_Retry()	Logger	Mounts the microSD card filesystem and retries if mounting fails.

Note: This table summarises the main firmware functions. Full source code is provided in the accompanying project files and development logbook.

4.3 Firmware Validation Strategy

The firmware was developed using staged validation to reduce integration risk. Each subsystem was tested independently before being combined into the final two-node system.

The Vehicle Dynamics Node was first tested by verifying communication with the MPU-6050 and running the IMU self-test. Normal IMU telemetry was then tested by reading accelerometer and gyroscope values and printing them over UART. CAN transmission was initially tested in loopback mode so that the STM32 could transmit and receive its own CAN frames before the physical CAN bus was connected.

The Logger Node was also developed in stages. The DS3231 RTC was tested by reading and printing a ticking timestamp over UART. The microSD card interface was tested by mounting the FAT32 filesystem and writing a simple text file before moving to CSV logging. The CSV logging function was first tested using fixed sample values before being connected to real CAN-received IMU data.

This staged approach made it easier to identify faults because each hardware and firmware block had already been validated before final integration. Final test evidence and results are presented later in Section 8.

5. CAN Communication Protocol

The telemetry system uses CAN to transfer IMU data from the Vehicle Dynamics Node to the Logger Node. CAN was selected because it supports structured message-based communication between distributed embedded nodes. In this prototype, the CAN bus carries accelerometer and gyroscope measurements from the Vehicle Dynamics Node to the Logger Node using standard 11-bit CAN identifiers.

The protocol was designed to keep each message short and simple. Since a classic CAN data frame can carry up to 8 data bytes, the six IMU values could not all be sent in one frame. The accelerometer and gyroscope readings were therefore transmitted as two separate CAN frames.

5.1 CAN Bus Configuration

Both STM32 nodes were configured to use the same CAN bitrate and frame format. The system used standard 11-bit CAN identifiers and data frames. Remote frames and extended identifiers were not used.

The CAN peripheral was configured for a bit rate of 500 kbit/s. This provided a suitable communication speed for transferring IMU telemetry between the two STM32 nodes while remaining straightforward to test on the prototype hardware.

The physical CAN bus was connected through SN65HVD230 transceivers on both nodes [5]. Each STM32 communicated with its transceiver using CAN TX and CAN RX logic signals, while the transceivers communicated with each other over the shared CANH and CANL differential pair [5].

Table V: CAN bus configuration used by both STM32 nodes.

Parameter	Value
CAN mode	Normal mode
Identifier type	Standard 11-bit ID
Frame type	Data frame
Bitrate	500 Kbit/s
Physical layer	SN65HVD230 transceivers
Bus lines	CANH and CANL
Termination	120 Ω at bus ends

5.2 CAN Frame Format

The MPU-6050 provides six main telemetry values: three accelerometer axes and three gyroscope axes. Each value is stored as a signed 16-bit integer. This gives a total of 12 bytes of IMU data per sample, before adding any sequence or status information. Since a classic CAN frame can only carry up to 8 bytes, the IMU sample was split into two frames.

The accelerometer frame uses CAN ID 0x120, while the gyroscope frame uses CAN ID 0x121. Each frame carries three signed 16-bit axis values, one sequence byte, and one status/reserved byte. The 16-bit values are transmitted in high-byte-first order.

Table VI: CAN message identifiers used for IMU telemetry.

CAN ID	Message Type	Source Node	Payload Summary
0x120	Accelerometer frame	Vehicle Dynamics	Accel X, Accel Y, Accel Z, sequence, status
0x121	Gyroscope frame	Vehicle Dynamics	Gyro X, Gyro Y, Gyro Z, sequence, status

Table VII: Payload format for accelerometer and gyroscope CAN frames.

Byte	Accelerometer Frame 0x120	Gyroscope Frame 0x121
0	Accel X high byte	Gyro X high byte
1	Accel X low byte	Gyro X low byte
2	Accel Y high byte	Gyro Y high byte
3	Accel Y low byte	Gyro Y low byte
4	Accel Z high byte	Gyro Z high byte
5	Accel Z low byte	Gyro Z low byte
6	Sequence number	Sequence number
7	Status/reserved	Status/reserved

This format allows each frame to remain within the 8-byte CAN payload limit while still carrying all three axes for the corresponding sensor type.

5.3 IMU Message Pairing

Each complete IMU sample is made from one accelerometer frame and one gyroscope frame. The Logger Node receives the two frame types separately, decodes their payloads, and stores the most recent accelerometer and gyroscope values. Once both frame types have been received, the Logger Node treats the data as one complete sample ready for timestamping and logging.

The sequence number byte is included so that the accelerometer and gyroscope frames from the same sample can be associated with each other. In the final prototype, the Logger Node primarily used the arrival of both frame types to decide when a complete sample was available. The sequence field provides a useful extension point for detecting missed, delayed, or mismatched frames in future versions.

A timestamp was not transmitted by the Vehicle Dynamics Node in the final two-node logging test. Instead, the Logger Node applied its own DS3231 RTC timestamp when writing the received sample to the CSV file. This kept the CAN protocol focused on sensor data and allowed the logger to control the final timing information stored in the log file.

6. Data Logging System

The Logger Node stores received telemetry data locally on a microSD card. After accelerometer and gyroscope CAN frames are decoded and paired, the firmware adds timing information and writes the complete sample to a CSV file. This allows the telemetry data to be removed from the embedded system after testing and analysed on a laptop.

The logging process used by the Logger Node is shown in Figure 8. Received CAN frames are decoded and combined

into complete IMU samples before timing information is added. Each sample is then formatted as a CSV row and written to the microSD card through FatFS and the custom SPI SD driver.

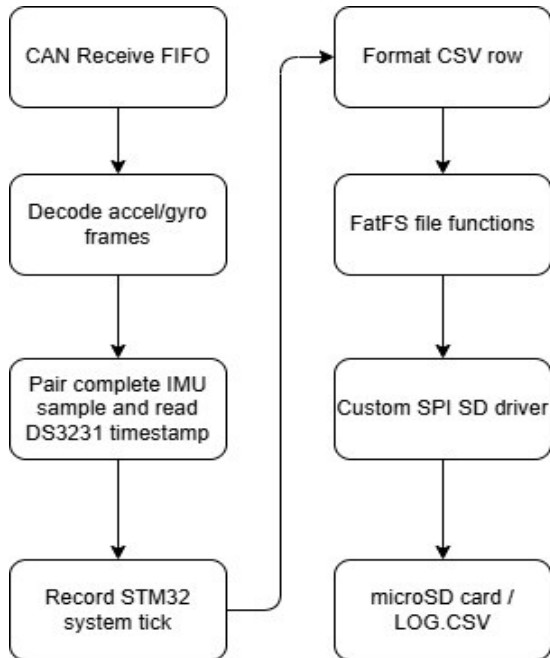


Figure 8: Logger Node data logging pipeline from CAN frame reception to CSV storage on the microSD card.

6.1 RTC Timestamping

The DS3231 real-time clock was used to provide real-world timestamps for logged telemetry rows [6]. The RTC communicates with the Logger Node over I2C and stores date and time values internally. When a complete IMU sample is ready to be logged, the firmware reads the RTC registers, converts the values into decimal format, and formats the timestamp as a readable string.

The timestamp format used in the CSV file was:

YYYY-MM-DD HH:MM: SS

An example timestamp is:

2026-07-12 16:30:05

Using an external RTC was useful because the STM32 system tick resets when the microcontroller restarts. The DS3231 provides a separate time source, allowing the generated log file to contain real date and time information rather than only relative time since boot.

6.2 microSD and FatFS Storage

The microSD card was used as removable local storage for the telemetry file. The Logger Node communicates with the microSD card over SPI using clock, MOSI, MISO, and chip select signals. A card-detect pin was also used so that the firmware could check whether a card was inserted before attempting to mount the filesystem.

FatFS was used to provide file-level access to the microSD card. This allowed the firmware to create, open, append to, and close files using functions such as `f_mount()`, `f_open()`, `f_write()`, and `f_close()`. A custom SPI-based SD card driver was used underneath FatFS to handle low-level communication with the SD card [7], [8].

During logging, the firmware opens LOG.CSV, writes a header if the file is empty, appends the new telemetry row, and then closes the file. This approach prioritised reliability during prototype testing because each row was finalised on the SD card after being written.

6.3 CSV Log Format

The final log format stores one complete IMU sample per row. Each row contains an RTC timestamp, a relative millisecond timestamp, three accelerometer values, three gyroscope values, and a status field.

Table VIII: CSV format used for logged IMU telemetry.

Column	Description
timestamp	Real-world timestamp from the DS3231 RTC.
time_ms	Relative time in milliseconds from HAL_GetTick().
accel_x	Raw X-axis accelerometer value.
accel_y	Raw Y-axis accelerometer value.
accel_z	Raw Z-axis accelerometer value.
gyro_x	Raw X-axis gyroscope value.
gyro_y	Raw Y-axis gyroscope value.
gyro_z	Raw Z-axis gyroscope value.
status	Sample status field.

The resulting CSV header was:

timestamp, time_ms, accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z, status

An example row is:

2026-07-12 16:30:05, 1234, 1532, -212, 15890, 40, -12, 3, OK

This structure was chosen because it keeps each accelerometer and gyroscope sample on the same row, making the file straightforward to inspect in Excel and import into MATLAB for later analysis.

7. MATLAB Post-Processing

After each logging test, the generated LOG.CSV file was transferred from the microSD card to a laptop for offline analysis. MATLAB was used to import the logged telemetry data, convert the raw IMU readings into physical units, estimate the orientation of the Vehicle Dynamics Node, and replay the motion using a simple 3D visualisation.

The MATLAB stage was included to demonstrate that the logged embedded data could be used for meaningful post-processing rather than only being stored as raw values. This completed the telemetry pipeline by connecting the STM32 data acquisition system to an offline analysis environment.

7.1 CSV Import and Unit Conversion

The CSV file was imported into MATLAB using `readtable()`. This allowed each column in the log file to be accessed by name, such as `time_ms`, `accel_x`, `accel_y`, `accel_z`, `gyro_x`, `gyro_y`, and `gyro_z`.

The relative time column was converted from milliseconds to seconds before plotting:

$$t = \text{time_ms} / 1000$$

The MPU-6050 was configured with an accelerometer range of ± 8 g and a gyroscope range of ± 250 degrees per second. Therefore, the raw accelerometer values were converted into units of g using:

$$\text{acceleration_g} = \text{raw_acceleration} / 4096$$

The raw gyroscope values were converted into degrees per second using:

$$\text{angular_velocity} = \text{raw_gyro} / 131$$

These converted values were then used for plotting and orientation estimation.

7.2 Roll and Pitch Estimation

Roll and pitch were estimated from the accelerometer data using the gravity vector. When the node is stationary or moving slowly, the accelerometer mainly measures the projection of gravity along its X, Y, and Z axes. As the board tilts, the measured gravity components change, allowing the tilt angle to be estimated.

The roll angle was estimated using the Y and Z accelerometer components:

$$\text{roll} = \text{atan2}(\text{ay}, \text{az})$$

The pitch angle was estimated using the X component and the combined Y-Z magnitude:

$$\text{pitch} = \text{atan2}(-\text{ax}, \sqrt{\text{ay}^2 + \text{az}^2})$$

In MATLAB, `atan2d()` was used so that the output angles were returned directly in degrees. This method provides a simple gravity-based estimate of roll and pitch. However, it is not a full sensor-fusion algorithm and becomes less accurate during rapid motion because the accelerometer also measures linear acceleration.

Yaw was not estimated in the final dashboard because the MPU-6050 does not provide an absolute heading reference. A magnetometer or other heading reference would be required for reliable yaw estimation.

7.3 Offline 3D Orientation Replay

An offline MATLAB dashboard was developed to replay the estimated orientation of the Vehicle Dynamics Node. A simple 3D board model was created using vertices and faces. For each logged sample, the calculated roll and pitch angles were converted into rotation matrices and applied to the board vertices. Updating the rotated vertices over time made the 3D model replay the measured tilt motion.

The dashboard included a 3D board display, a roll/pitch plot, a moving time marker, a time slider, and play/pause controls. This allowed the logged motion to be inspected manually or replayed automatically after the test.

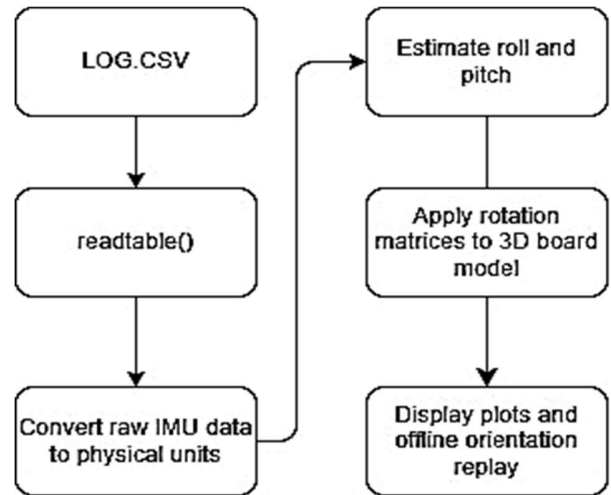


Figure 9: MATLAB post-processing flow from CSV import to roll/pitch estimation and offline 3D orientation replay.

The replay was used as a qualitative validation tool. By comparing the MATLAB 3D replay with a video of the physical board being tilted, it was possible to confirm that the logged IMU data produced motion behaviour that matched the real prototype movement.

8. Testing and Results

Testing was carried out incrementally so that each subsystem could be verified before the complete two-node telemetry system was tested. This reduced integration risk by confirming that the IMU, CAN communication, RTC timestamping, SD card logging, and MATLAB analysis stages were each operating correctly before being combined.

8.1 MPU-6050 Self-Test Verification

The first test stage verified communication with the MPU-6050 and checked that the sensor could respond to register-level commands. The STM32 communicated with the IMU over I2C and performed the MPU-6050 self-test procedure. This confirmed that the sensor was detected correctly and that the firmware could write to and read from the required configuration and self-test registers.

```
Gyro self-test change:
X = 3.22%
Y = 9.79%
Z = 5.31%
Accel self-test change:
X = 3.06%
Y = 2.34%
Z = 8.17%
```

UART output from the MPU-6050 self-test on the Vehicle Dynamics Node, showing the calculated accelerometer and gyroscope self-test percentage changes.

The measured self-test percentage changes were within the acceptable range specified in the MPU-6050 documentation (between -14% and 14%). This confirmed that all accelerometer and gyroscope axes produced a measurable self-test response before normal telemetry operation [3], [4].

The self-test stage was important because it verified the IMU interface before normal telemetry acquisition was attempted. Once the self-test passed, the MPU-6050 was configured for normal accelerometer and gyroscope measurement.

The percentage change was calculated by comparing the measured self-test response with the factory trim values obtained from the MPU-6050 self-test registers. The full calculation procedure and code listing are provided in the development logbook.

8.2 Normal IMU Telemetry Verification

After the self-test stage, the MPU-6050 was configured for normal operation. The Vehicle Dynamics Node read accelerometer and gyroscope registers over I2C and printed the raw sensor values over UART.

When the board was stationary, one accelerometer axis showed a value close to the expected 1 g gravity component, while the remaining axes were closer to zero depending on the board orientation. When the board was moved, the accelerometer and gyroscope values changed accordingly.

This test verified the sensor acquisition stage of the project. It confirmed that the Vehicle Dynamics Node could read motion data before the values were packaged into CAN frames.

8.3 CAN Loopback Verification

Before connecting the two physical nodes, CAN communication was tested using loopback mode on the STM32. In loopback mode, the CAN peripheral transmits frames internally and receives them back without requiring a second node on the bus. This allowed CAN frame packing, transmission, reception, and decoding to be verified in isolation.

The accelerometer and gyroscope values were packed into CAN frames using IDs 0x120 and 0x121. The received loopback frames were decoded and printed over UART to confirm that the transmitted values matched the received values.

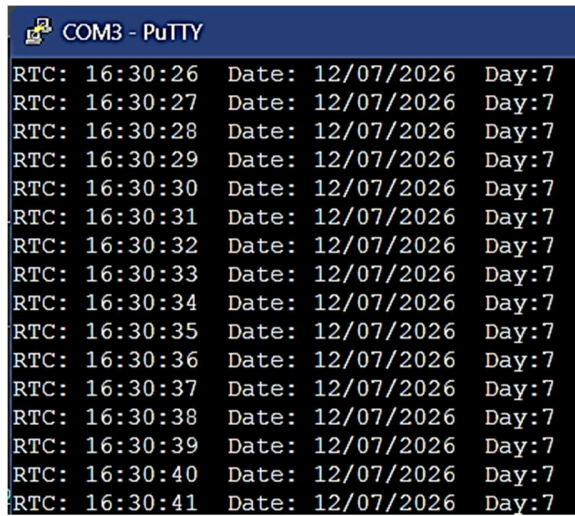
```
COM3 - PuTTY
RX Gyro | X=-525 Y=-102 Z=-486 | Seq=83
RX Time | T=34727 ms | Seq=83
RX Accel | X=-646 Y=-1132 Z=-4699 | Seq=103
RX Gyro | X=-435 Y=285 Z=-412 | Seq=103
RX Time | T=35127 ms | Seq=103
RX Accel | X=-644 Y=-1014 Z=-4614 | Seq=123
RX Gyro | X=-663 Y=560 Z=-528 | Seq=123
RX Time | T=35527 ms | Seq=123
RX Accel | X=-549 Y=-1000 Z=-4654 | Seq=143
RX Gyro | X=-1261 Y=-499 Z=-1126 | Seq=143
RX Time | T=35926 ms | Seq=143
RX Accel | X=-445 Y=-1083 Z=-4555 | Seq=163
RX Gyro | X=-925 Y=22 Z=-204 | Seq=163
RX Time | T=36326 ms | Seq=163
RX Accel | X=-851 Y=-905 Z=-4669 | Seq=183
RX Gyro | X=-483 Y=-75 Z=120 | Seq=183
RX Time | T=36726 ms | Seq=183
RX Accel | X=-687 Y=-1101 Z=-4779 | Seq=203
RX Gyro | X=-381 Y=548 Z=709 | Seq=203
RX Time | T=37126 ms | Seq=203
RX Accel | X=-689 Y=-1046 Z=-4658 | Seq=223
RX Gyro | X=-755 Y=-10 Z=-433 | Seq=223
RX Time | T=37526 ms | Seq=223
```

Figure 11: CAN loopback verification showing accelerometer and gyroscope frames transmitted and decoded by the STM32.

This test confirmed that the CAN frame format was correct before the system was moved to a physical two-node CAN bus.

8.4 RTC Timestamp Verification

The DS3231 RTC was tested independently on the Logger Node before it was used for telemetry logging. The STM32 read the RTC registers over I2C, converted the BCD-encoded values into decimal format, and printed the resulting timestamp over UART.



RTC	Date	Day
16:30:26	12/07/2026	Day:7
16:30:27	12/07/2026	Day:7
16:30:28	12/07/2026	Day:7
16:30:29	12/07/2026	Day:7
16:30:30	12/07/2026	Day:7
16:30:31	12/07/2026	Day:7
16:30:32	12/07/2026	Day:7
16:30:33	12/07/2026	Day:7
16:30:34	12/07/2026	Day:7
16:30:35	12/07/2026	Day:7
16:30:36	12/07/2026	Day:7
16:30:37	12/07/2026	Day:7
16:30:38	12/07/2026	Day:7
16:30:39	12/07/2026	Day:7
16:30:40	12/07/2026	Day:7
16:30:41	12/07/2026	Day:7

Figure 12: DS3231 RTC verification showing timestamp values read by the Logger Node.

The UART output showed the timestamp increasing once per second, confirming that the DS3231 registers were being read correctly and that the BCD-to-decimal conversion was working.

This test confirmed that the Logger Node could obtain real-world date and time information. The RTC was then used to timestamp each complete IMU sample written to the CSV log file.

8.5 microSD and CSV Logging Verification

The microSD card interface was verified before receiving real CAN data. The Logger Node checked the card-detect pin, mounted the FAT32 filesystem using FatFS, and wrote a test file to confirm that the SPI SD card driver and filesystem bridge were working correctly.

After the basic file test, the CSV logging function was tested using fixed sample values. Successful verification required the SD card to be detected, the FAT32 filesystem to mount, a file to be created or opened, data to be written, and the file to be readable on a laptop after removal from the prototype.

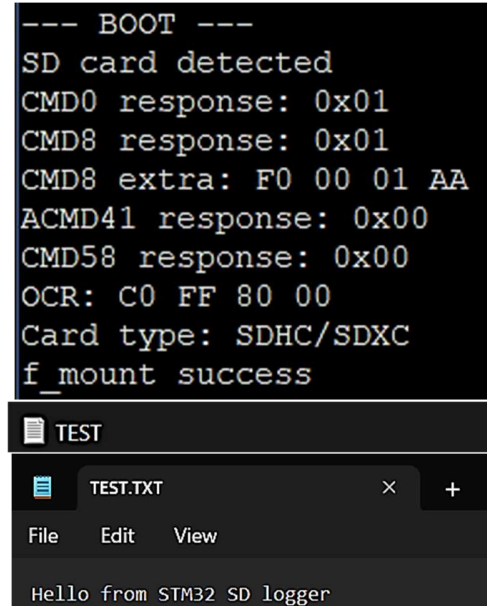


Figure 13: microSD and FatFS verification showing successful filesystem mounting and file writing.

The SD logging test verified the SPI card initialisation and FAT filesystem access described in the SD card documentation [7], [8].

8.6 Final Two-Node CAN-to-SD Integration

After the individual subsystems had been verified, the Vehicle Dynamics Node and Logger Node were connected using the physical CAN bus, SN65HVD230 transceivers and 120 Ω bus termination, following the transceiver application guidance [5]. The Vehicle Dynamics Node transmitted accelerometer and gyroscope frames over CAN. The Logger Node received both frame types, decoded the payloads, paired the accelerometer and gyroscope data, added a DS3231 timestamp, and wrote the complete sample to LOG.CSV.


```

COM3 - PuTTY
RX GYRO: -423,-182,90
RX ACCEL: 74,-54,3762
CSV row written: 54 bytes
RX GYRO: -427,-190,87
RX ACCEL: 75,-56,3740
CSV row written: 54 bytes
RX GYRO: -429,-174,92
RX ACCEL: 70,-55,3739
CSV row written: 54 bytes
RX GYRO: -425,-165,101
RX ACCEL: 82,-50,3753
CSV row written: 55 bytes
RX GYRO: -427,-178,89
RX ACCEL: 72,-38,3758
CSV row written: 54 bytes
RX GYRO: -428,-182,93
RX ACCEL: 69,-49,3755
CSV row written: 55 bytes
RX GYRO: -426,-177,97
RX ACCEL: 72,-61,3754
CSV row written: 55 bytes
RX GYRO: -430,-184,90
RX ACCEL: 71,-45,3750

```

Figure 14: Final two-node CAN-to-SD integration test showing CAN reception and successful CSV row writing.

The repeated RX ACCEL, RX GYRO, and CSV row written messages confirmed that the complete embedded telemetry path was operating correctly. This verified that real IMU data was being transmitted over the physical CAN bus and stored on the microSD card by the Logger Node.

12/07/2026 16:30	9070	74	-54	3762	-423	-182	90
12/07/2026 16:30	9295	75	-56	3740	-427	-190	87
12/07/2026 16:30	9520	70	-55	3739	-429	-174	92
12/07/2026 16:30	9745	82	-50	3753	-425	-165	101
12/07/2026 16:30	9970	72	-38	3758	-427	-178	89
12/07/2026 16:30	10196	69	-49	3755	-428	-182	93
12/07/2026 16:30	10421	72	-61	3754	-426	-177	97

Figure 15: Logged LOG.CSV file containing timestamped accelerometer and gyroscope data received over CAN.

The generated CSV file confirmed that each logged row contained a timestamp, relative time, accelerometer values, gyroscope values, and status field in the expected format.

8.7 MATLAB Post-Processing and Replay

The final verification stage tested whether the generated LOG.CSV file could be used for offline analysis. The CSV

file was imported into MATLAB, where the raw IMU values were converted into physical units, plotted over time, and used to estimate roll and pitch.

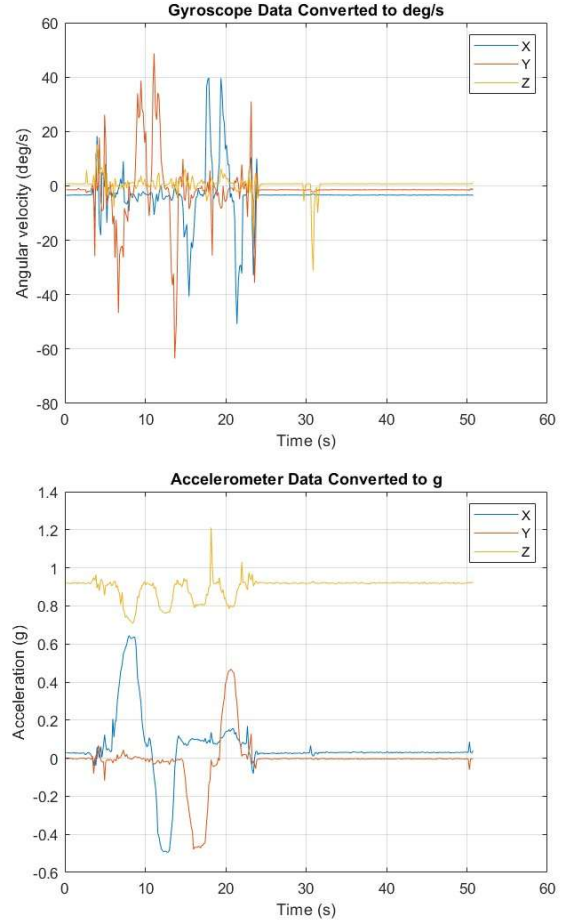


Figure 16: MATLAB plot generated from the logged IMU telemetry file.

An offline 3D dashboard was then used to replay the estimated orientation of the Vehicle Dynamics Node. The replay was compared with a video of the physical board being tilted during the same logging test. The replay comparison is shown in Fig. 17.

The MATLAB replay showed the same general left/right and forward/back tilt behaviour as the physical prototype. This provided qualitative validation of the full telemetry chain, from physical motion to IMU acquisition, CAN transmission, RTC timestamping, CSV logging, and offline visualisation.

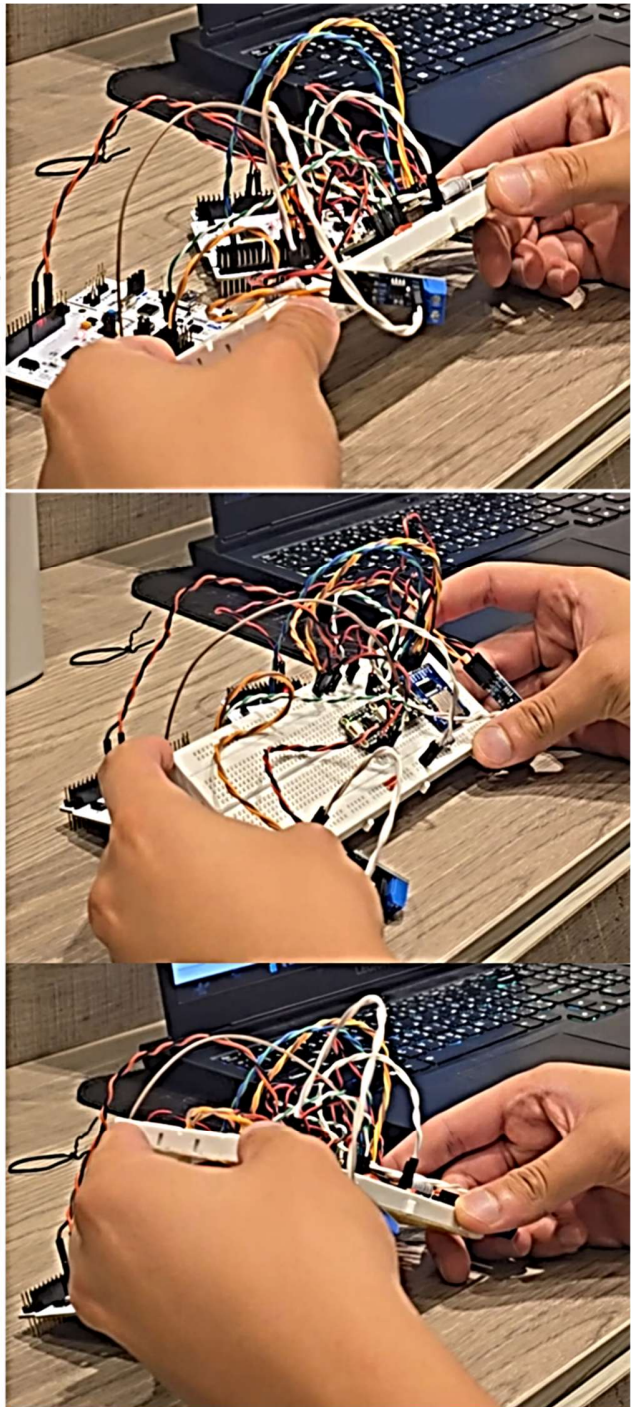
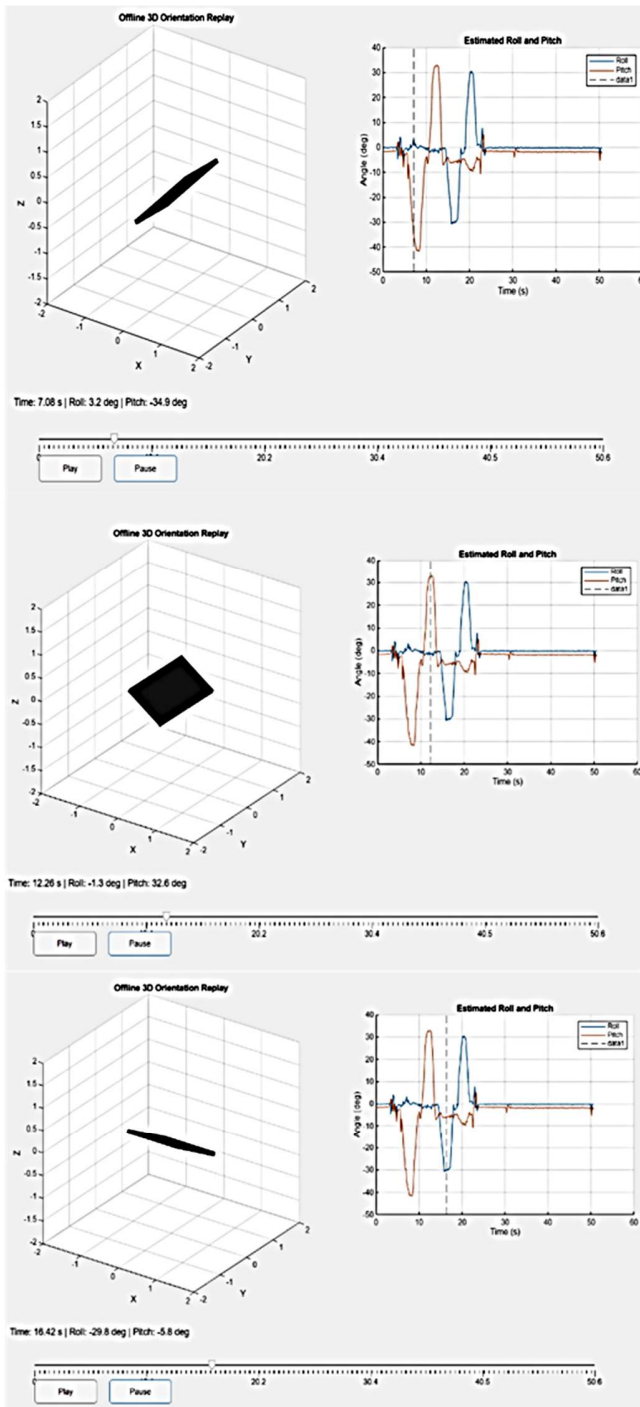


Figure 17: MATLAB offline orientation replay used to validate the logged motion against the physical board movement.

8.8 End-to-End Logging Rate

The MPU-6050 was configured with a sample rate divider of 4, giving an IMU sample rate of approximately 200 Hz. In the final Vehicle Dynamics Node firmware, the CAN transmission was intentionally reduced by sending one telemetry sample every 20 IMU samples. This produced an effective transmitted telemetry rate of approximately 10 complete IMU samples per second.

Each complete IMU sample was transmitted using two CAN frames: one accelerometer frame with CAN ID 0x120 and one gyroscope frame with CAN ID 0x121. Therefore, the final logged telemetry rate corresponded to approximately 10 CSV rows per second and 20 CAN frames per second.

This rate was selected to prioritise reliable end-to-end logging to the microSD card during prototype testing rather than maximising CAN bus throughput.

Table IX: End-to-end telemetry rate during final integration testing.

Parameter	Value
MPU-6050 configured sample rate	200 samples/s
CAN transmission decimation	1 in every 20 IMU samples
Effective logged IMU rate	Approximately 10 samples/s
CAN frames per complete sample	2 frames
Effective CAN frame rate	Approximately 20 frames/s
Logged output	Approximately 10 CSV rows/s

9. Limitations

Although the prototype successfully demonstrated the full telemetry pipeline, several limitations remain. The system was developed as a controlled bench prototype rather than a vehicle-ready product. The hardware was assembled using breadboards and jumper wires, which made development and debugging easier but limited the mechanical robustness of the system. A vehicle-ready implementation would require soldered wiring or a PCB, secure connectors, strain relief, protected power input, and an enclosure.

The orientation estimation was also limited by the sensor used. The MPU-6050 provides accelerometer and gyroscope data but does not include a magnetometer. As a result, roll and pitch could be estimated from the accelerometer gravity vector, but absolute yaw could not be measured reliably.

Gyroscope-only yaw estimation was not used because it would drift over time without an external heading reference.

The roll and pitch estimates were suitable for slow manual motion and qualitative replay, but they were not precision-calibrated. During rapid movement or vibration, the accelerometer measures both gravity and linear acceleration, which can reduce the accuracy of gravity-based tilt estimation. Therefore, the MATLAB replay was used as a qualitative visualisation tool rather than a calibrated attitude measurement system.

The final logging rate was intentionally limited to prioritise reliable SD card writing. The system logged approximately 10 complete IMU samples per second, which was sufficient for the controlled validation tests. Higher data rates would require improved buffering, fewer file open/close operations, or a faster storage strategy.

Finally, the system was tested using controlled manual movement rather than on a real vehicle. This verified the embedded telemetry chain, but further testing would be required under vibration, longer cable lengths, electrical noise, and realistic mounting conditions.

10. Future Work

Several improvements could be made to extend the telemetry system beyond the current prototype. The first improvement would be to replace the breadboard wiring with a more robust hardware implementation. A soldered stripboard or custom PCB would improve mechanical reliability, reduce loose connections, and make the system more suitable for longer tests. Secure connectors, strain relief, a protected power input, and an enclosure would also be required for a more practical installation.

The sensing capability could also be expanded by adding more CAN-based nodes. A thermal monitoring node could be added to transmit temperature measurements to the Logger Node. A wheel speed or GPS node could also be included to provide speed and distance data, which cannot be obtained reliably from accelerometer integration alone due to drift.

The orientation estimation could be improved by adding a magnetometer. This would provide an external heading reference and allow yaw estimation to be added to the MATLAB dashboard. A future version could also implement sensor fusion using accelerometer, gyroscope, and magnetometer data to improve the stability of the orientation estimate.

The data logging system could be improved by increasing the sustained logging rate. In the current prototype, the logging rate was intentionally limited to prioritise reliable

SD card writing. A future implementation could buffer samples in RAM, keep the CSV file open during acquisition, reduce repeated file open and close operations, or use a faster storage interface.

Finally, the MATLAB analysis could be extended into a more complete telemetry dashboard. Additional plots, replay controls, event markers, and automated summary statistics could be added. The system could also be tested over longer durations and under more realistic vibration and electrical noise conditions to better evaluate its robustness.

11. Conclusion

This report presented the design and implementation of a modular STM32-based CAN telemetry system for IMU data acquisition, timestamped local logging, and offline analysis. The system was built around two STM32 NUCLEO-F446RE development boards: A Vehicle Dynamics Node for acquiring accelerometer and gyroscope data from an MPU-6050 IMU, and a Logger Node for receiving CAN frames, applying RTC timestamps, and storing telemetry data on a microSD card.

The project successfully demonstrated an end-to-end embedded telemetry pipeline. IMU data was acquired over I2C, transmitted over a physical CAN bus, received and decoded by the Logger Node, timestamped using a DS3231 RTC, and written to LOG.CSV in a format suitable for MATLAB analysis. The firmware was verified through staged testing, including MPU-6050 self-test, normal IMU telemetry, CAN loopback, RTC timestamp verification, SD card file writing, and final two-node CAN-to-SD integration.

The logged data was imported into MATLAB, converted into physical units, plotted, and used to estimate roll and pitch orientation. An offline 3D dashboard was developed to replay the motion of the Vehicle Dynamics Node, and the replay showed qualitative agreement with the physical board movement during testing.

Although the prototype was not designed as vehicle-ready hardware, it achieved its main objective of demonstrating a complete modular telemetry chain from sensing to communication, storage, and offline visualisation. Future improvements could include a more robust PCB-based implementation, additional CAN sensor nodes, magnetometer-assisted yaw estimation, higher logging rates, and testing under more realistic vehicle conditions.

12. References

- [1] STMicroelectronics, “STM32 Nucleo-64 boards,” Data brief DB2196, Rev. 22, Feb. 2026.
- [2] STMicroelectronics, “STM32F446xx advanced Arm-based 32-bit MCUs,” Reference manual RM0390, Rev. 9, Feb. 2026.
- [3] InvenSense, “MPU-6000 and MPU-6050 Product Specification,” Rev. 3.4, Aug. 2013.
- [4] InvenSense, “MPU-6000 and MPU-6050 Register Map and Descriptions,” Rev. 4.2, Aug. 2013.
- [5] Texas Instruments, “SN65HVD230, SN65HVD231, SN65HVD232 3.3-V CAN Transceivers,” Datasheet SLOS346G, Mar. 2001, revised Jun. 2002.
- [6] Maxim Integrated, “DS3231 Extremely Accurate I2C-Integrated RTC/TCXO/Crystal,” Datasheet, Rev. 10, Mar. 2015.
- [7] Adafruit Industries, “Micro SD Card Breakout Board Tutorial,” May 2025.
- [8] SD Card Association, “SD Specifications Part 1 Physical Layer Simplified Specification,” Version 6.00, Apr. 2017.